

1.

**Engineering, Built Environment and IT
Department of Computer Science**

**Programming Languages
COS 333**

**Semester Test 1
25 March 2025**

Examiner: Mr W. S. van Heerden

Instructions:

1. Read the questions carefully and answer all questions.
2. This section comprises of **21** questions, and a total of **35** marks.
3. You have **1.5** hours to complete this test.
4. This test is an *online* assessment:
5. The test will automatically submit when the time (1.5 hours) expires. You can also submit the test yourself if you are done ahead of time.
6. This test is **closed book** and is subject to the University of Pretoria integrity statement provided below.
 - You are not allowed to consult any sources other than the Practical 2 specification and the MIT/GNU Scheme Reference Manual (provided as documentation for Practical 2).
 - For practical implementation questions, your Scheme submission must be working Scheme code that will successfully interpret using version 8.8 of the DrRacket interpreter. You may (and should) use this interpreter to test your submissions. Once you have finished writing each program, make sure each is saved in a text file with the appropriate name (described in the question), and submit it to the correct upload slot.
 - You may use a non-programmable calculator.
 - You may not use any other programmable devices.

Integrity Statement:

The University of Pretoria commits itself to producing academic work of integrity. I affirm that I am aware of and have read the Rules and Policies of the University, more specifically the Disciplinary Procedure and the Tests and Examinations Rules, which prohibit any unethical, dishonest or improper conduct during tests, assignments, examinations and/or any other forms of assessment. I am aware that no student or any other person may assist or attempt to assist another student, or obtain help, or attempt to obtain help from another student or any other person during tests, assessments, assignments, examinations and/or any other forms of assessment.

Question 1

2 Points

Imagine that a recent graduate joins a company as a software tester who will be providing high- and low-level feedback on software errors, but will not be developing new systems. The graduate has learned Java during their studies. The company uses Python in its projects, and considers execution performance to be very important. **Identify** two reasons for studying concepts of programming languages, which are applicable to the graduate. You do not need to justify your answers.

Reason 1:

Reason 2:

Question 2

1 Point

Assume two hypothetical programming languages, A and B, both of which support primitive numeric types and functions. Programming language A allows functions to return only numeric types. Programming language B allows functions to return numeric types and functions. Consider the following statements, and **select** the one that is the most correct.

- ☐ Programming language A is more orthogonal than programming language B because programming language A supports a smaller set of constructs than programming language B.
- ☐ Programming language A is more orthogonal than programming language B because returning functions makes no sense.
- ☐ Programming language B is more orthogonal than programming language A because programming language B supports a larger set of constructs than programming language A.
- ☐ Programming language B is more orthogonal than programming language A because programming language B allows more legal return values than programming language A.

Question 3

1 Point

Some programming languages use special words to mark the start and end of a compound statement (i.e. a block). Consider the following implications, and **select** the one that most correctly describes a consequence of using special words to mark compound statements.

- ☐ Readability is decreased.
- ☐ Writability is improved.
- ☐ Writability is decreased.
- ☐ Reliability is improved.

Question 4

1 Point

Many programming languages support some kind of aliasing. Consider the following implications, and **select** the one that most correctly describes a consequence of including support for aliasing in a programming language.

- ☐ Readability is improved.
- ☐ Writability is improved.
- ☐ Reliability is increased.
- ☐ The overall cost of the programming language decreases.

Question 5

1 Point

Type checking is generally considered to be beneficial to reliability. Consider the following statements about type checking, and **select** the one that is the most correct.

- ☐ Type checking increases the cost of execution for a programming language.
- ☐ Type checking increases the cost of reliability for a programming language.
- ☐ Type checking increases the writability of a programming language, because it gives the programmer more flexibility.
- ☐ Type checking has no impact on any of the programming language evaluation criteria.

Question 6

1 Point

Consider the following programming domains, and **select** the one that a hybrid implementation system is most suitable for.

- ☐ Scientific applications.
- ☐ Business applications.
- ☐ Systems programming.
- ☐ Web software.

Question 7

2 Points

Scheme has poor execution performance. Consider the following factors, and **select** only those that contribute to the poor performance of Scheme. Note that incorrectly selected options will be penalised.

- ☐ The implementation method that is most often used for Scheme.
 - ☐ The programming domain that Scheme was designed for.
 - ☐ The programming methodology that is most often used with Scheme.
 - ☐ Modern computer architecture.
- Select up to 4 options

Question 8

1 Point

Various factors can influence programming language design. Consider the following programming language design influences, and **select** the one that had its earliest impact on the design of Smalltalk.

- ☐ Computer architecture.
 - ☐ Programmer efficiency becoming more important than machine efficiency.
 - ☐ The move from process oriented programming to data oriented programming.
 - ☐ The move to object-oriented programming.
-

Question 9

1 Point

Consider the following statements about Fortran 90, and **select** the one that is most correct.

- ☐ Fortran 90 was only focused on the execution speed of compiled program code.
 - ☐ Fortran 90 introduced language features that improved writability at the expense of the execution speed of compiled program code.
 - ☐ Fortran 90 introduced language features that reduced writability in order to improve the execution speed of compiled program code.
 - ☐ Fortran 90 required code to be structured in a way that facilitated code execution from punch cards.
-

Question 10

1 Point

Consider the following statements about the successes of ALGOL 60, and **select** the one that is most correct.

- ☐ ALGOL 60 was successful as a programming language that was applicable to many different programming domains.
- ☐ ALGOL 60 was completely successful as a machine-independent programming language.
- ☐ ALGOL 60 served as a very readable standard for documenting algorithms.
- ☐ ALGOL 60 formal syntax description that was widely accepted at the time ALGOL 60 was created.

Question 11

1 Point

Consider the following statements about PL/I, and **select** the one that is most correct.

- ☐ PL/I was intended for scientific applications.
- ☐ PL/I aimed to be more well-defined than programming languages developed in previous years.
- ☐ PL/I aimed to have better generality than programming languages developed in previous years.
- ☐ PL/I aimed to have better portability than programming languages developed in previous years.

Question 12

1 Point

Consider the following statements about APL, and **select** the one that is most correct.

- ☐ APL demonstrates that there is not necessarily a tradeoff between readability and writability, because it has both good readability and good writability.
- ☐ APL demonstrates that there is often a tradeoff between readability and writability, because APL has good readability and poor writability.
- ☐ APL demonstrates that there is often a tradeoff between readability and writability, because APL has poor readability and good writability.
- ☐ APL demonstrates that there is not necessarily a tradeoff between readability and writability, because it has both poor readability and poor writability.

Question 13

2 Points

Consider the following language features, and **select** only those that were pioneered by Simula 67. Note that incorrectly selected options will be penalised.

- ☐ Systems programming support.
 - ☐ Coroutines.
 - ☐ Classes and objects.
 - ☐ Inheritance and polymorphism.
- Select up to 4 options

Question 14

1 Point

Consider the following statements about Delphi, and **select** the one that is most correct.

- ☐ Delphi is an imperative programming that was extended with support for object-oriented programming.
- ☐ Delphi is a purely object-oriented programming language.
- ☐ Delphi has no support for object-oriented programming.
- ☐ Delphi is less reliable than C++.

Question 15

1 Point

Consider the following statements about Ruby, and **select** the one that is most correct.

- ☐ Ruby is a compiled programming language.
- ☐ Ruby is primarily an imperative programming language with optional support for object-oriented programming.
- ☐ Ruby supports classes and objects, but does not support inheritance and polymorphism.
- ☐ Ruby is unreliable because its classes and objects can have different structures at different times.

Question 16

2 Points

Consider the following programming language features, and **select** only those that improve the orthogonality of Scheme. Note that incorrectly selected options will be penalised.

- ☐ The fact that only symbolic and numeric atoms are supported.
- ☐ The fact that functions and lists are represented the same way.
- ☐ The fact that functions can be used as parameters.
- ☐ The fact that functional side effects are not allowed.

Select up to 4 options

Question 17

2 Points

Consider the following Scheme code:

```
(doSomething thing)
```

Consider the following options, and **select** only those that describe what `thing` can be. Note that incorrectly selected options will be penalised.

- ☐

A symbolic atom.
 - ☐

A name to which a value has been bound, and for which the binding can change at a later time.
 - ☐

A name to which a value has been bound, and for which the binding cannot change.
 - ☐

The name of a function.
- Select up to 4 options

Question 18

2 Points

One way that referential transparency can break down is through functional side effects. Consider the following scenarios, and **select** the the options that describe a different way in which referential transparency could break down, which is **not** a functional side effect. Note that incorrectly selected options will be penalised.

- ☐

When a variable is used in a function.
 - ☐

When a function returns no value.
 - ☐

When a function returns a random value.
 - ☐

When a function reads a user input, and returns this value.
- Select up to 4 options

Question 19

1 Point

Consider the following Scheme code:

```
(eval '(+ 2 3))
```

Consider the following options, and **select** the one that most correctly explains what happens when the above program code is executed.

- ☐

The code fails to execute because the `eval` function is undefined.
- ☐

The Scheme interpreter evaluates the list `(+ 2 3)` as if it is a function application, producing the result 5.
- ☐

The function application `(+ 2 3)` is evaluated, producing the result 5. The function `eval` prints out this result.
- ☐

The function application `(+ 2 3)` is evaluated, producing the result 5. The result is converted into the list `(5)`. The function `eval` prints out the content of this list, which is the value 5.

Question 20

5 Points

Write a Scheme function named `coneArea`, which receives two numeric parameters. The first parameter is the radius of the circular base of a cone. The second parameter is the slant height of the cone. The function should yield (not print out) the area of the cone, calculated according to the following equation:

where r is the radius of the base, and l is the slant height of the cylinder. The function should yield a zero result if either the radius or the slant height is negative. Use a `let` (not one or more `define` functions) to bind the the result of 22 divided by 7 to the name `pi` that represents π (note that `22/7` is a fraction literal in Scheme, not a division), and the result of r^2 to the name `rSquared`.

For example, the function application

```
(display (coneArea 1.2 2.1))
```

should yield a result of approximately 12.445714285714285.

Submit your program code in a file named `u99999999.scm` (where 99999999 is your student number) to the assignment submission slot labelled "Semester Test 1: Arithmetic Scheme Function", and select the "True" answer below once you have done so. If you do not make a submission, select the "False" answer below.

Take careful note of the following requirements:

- Your submission must be working Scheme code that will be successfully interpreted by version 8.8 of the DrRacket interpreter (this means, amongst other things, that lowercase letters should be used for all built-in function names). You may (and should) use the DrRacket interpreter to test your submission.
- You may define additional helper functions.
- Your Scheme function may only use the following built-in functions in the way they are used in the slides and textbook (failure to do so will result in a zero mark for this question):
- Function construction: `lambda`, `define`
- Binding: `let`
- Arithmetic: `+`, `-`, `*`, `/`, `abs`, `sqrt`, `remainder`, `modulo`, `min`, `max`
- Boolean values: `#t`, `#f`
- Equality predicates: `=`, `>`, `<`, `>=`, `<=`, `even?`, `odd?`, `zero?`, `negative?`, `eqv?`, `eq?`
- Logical predicates: `and`, `or`, `not`
- List predicates: `list?`, `null?`
- Conditionals: `if`, `cond`, `else`
- Quoting: `quote`, `'`
- List manipulation: `list`, `car`, `cdr`, `cons`
- Input and output: `display`, `printf`, `newline`, `read`

True

False

Question 21

5 Points

Write a Scheme function named `countDivisors`, which is applied to two numeric parameters. The first parameter is a numeric atom called `atm` and the second parameter is a simple numeric list (i.e. a list containing only numeric atoms) called `lst`. The function should yield (not print out) the number of divisors of `atm` contained in `lst`. A divisor of the value `a` is a numeric value that divides perfectly into `a` (for example, 3 is a divisor of 6).

For example, the function application

```
(countDivisors 6 '())
```

should yield 0, because the second parameter contains no divisors of 6. As another example, the function application

```
(countDivisors 6 '(4 12))
```

should also yield 0, because 4 and 12 are not divisors of 6. As a final example, the function application

```
(countDivisors 6 '(1 4 3 12))
```

should yield 2 because only 1 and 3 are divisors of 6.

Submit your program code in a file named `u99999999.scm` (where 99999999 is your student number) to the assignment submission slot labelled "Semester Test 1: List Processing Scheme Function", and select the "True" answer below once you have done so. If you do not make a submission, select the "False" answer below.

Take careful note of the following requirements:

- Your submission must be working Scheme code that will be successfully interpreted by version 8.8 of the DrRacket interpreter (this means, amongst other things, that lowercase letters should be used for all built-in function names). You may (and should) use the DrRacket interpreter to test your submission.
- You may define additional helper functions.
- Your Scheme function may only use the following built-in functions in the way they are used in the slides and textbook (failure to do so will result in a zero mark for this question):
- Function construction: `lambda`, `define`
- Binding: `let`
- Arithmetic: `+`, `-`, `*`, `/`, `abs`, `sqrt`, `remainder`, `modulo`, `min`, `max`
- Boolean values: `#t`, `#f`
- Equality predicates: `=`, `>`, `<`, `>=`, `<=`, `even?`, `odd?`, `zero?`, `negative?`, `eqv?`, `eq?`
- Logical predicates: `and`, `or`, `not`
- List predicates: `list?`, `null?`
- Conditionals: `if`, `cond`, `else`
- Quoting: `quote`, `'`
- List manipulation: `list`, `car`, `cdr`, `cons`
- Input and output: `display`, `printf`, `newline`, `read`

True

False